QUANT SINGULARITY

AI Research Engineer — Intern Screening Project

Summer 2026 intake · AI-SLM track

Context

Quant Singularity is building an AI-supervised systematic trading platform for Indian derivatives markets. The AI-SLM track owns the intelligence layer: small language models fine-tuned on structured market state data, grounded with relevant historical context through retrieval-augmented generation, and connected to downstream systems through an orchestration layer that governs when model outputs are acted on and when they are not.

This project is the final screening stage for the AI Research Engineer intern role. The brief below is designed to show us how you think, not only how you code. We care far more about the rigour of your design choices and the honesty of your evaluation than the headline accuracy of your model. Read the entire document before you start. Several design choices are explicitly left to you and will be assessed on their defensibility.

The project

You are building a signal pod — a small language model that looks at a snapshot of NIFTY options market data and outputs a trading signal. That signal then passes through an orchestrator, a lightweight wrapper that decides whether the signal is safe to send downstream or should be suppressed.

The pod's output is read by automated systems, not people. This means the output must be valid, structured JSON on every single call. A pod that produces high-conviction directional signals in market conditions it has never seen in training is not a research artefact that missed a corner case. It is a liability.

The schema is fixed: a direction (CE, PE, or NEUTRAL), a conviction score between 0.0 and 1.0, a horizon (intraday or next_session), a signal ID, and a timestamp. If the model fails to produce valid JSON, the pod must return NEUTRAL with conviction 0.0 and log what went wrong. This fallback is not optional — it must be built and tested.

One thing to think carefully about: the conviction field is a number your model generates as text. It is not a softmax probability. In your report, explain where this value actually comes from in your fine-tuned model, why softmax over the next token is not the right answer, and what you did to make it meaningful.

The orchestrator wraps the pod and applies three rules in sequence. If ADX is below 20, suppress the pod entirely and return NEUTRAL without calling the model. If the model output fails to parse, return NEUTRAL and log the raw output. If conviction is below 0.40, downgrade direction to NEUTRAL. Every decision must be logged with a reason code and the values that triggered it. The downstream pipeline only ever reads the orchestrator output — never the raw pod signal.

You will also run a RAG experiment. A retrieval function is provided — `retrieve(market_state, k=3)` — which returns the three most similar historical market episodes from a corpus. You do not need to build the retrieval layer. Your job is to design the prompt template that uses those retrieved episodes as context, run the experiment with and without retrieval across your evaluation set, and report honestly whether it helped and why.

A deliberate note on the training data: we have not documented the quirks of the instruction-format file. Treat it as you would any dataset arriving from a real third-party source on your first day. Not all examples are safe to use as provided. Part of your job is to figure out what is in it, what is wrong with it, and what to do about it.

Specification

Parameter	Specification
Asset	NIFTY 50 options signal pod. Market state features: spot price, ATM IV, IV skew, PCR, ADX, realised volatility, India VIX, DTE, moneyness band.
Stack	Python (mandatory). Fine-tuning: Kaggle free-tier GPU (T4, 30 hrs/week) via LoRA. Inference: CPU, 4-bit quantization. Experiment tracking: MLflow from the first run.
Base model	TinyLlama-1.1B or Phi-2 — your choice. Justify it in Section 3.
Fine-tuning method	LoRA only. Rank in the range 4–16. Justify your choice.
Compute	All model training must be run on Kaggle free-tier GPU. No other GPU compute is required or permitted. Inference endpoint must run on CPU.
Evaluation	Walk-forward only. Days 31–60, evaluated in 5-day rolling blocks. k-fold on a time series is disqualifying.
Experiment tracking	MLflow from the first run. Retrofitted tracking will be detected. Kaggle notebook URL must appear in the repo README.
Timeline	5 calendar days from brief release. Plan for 15 to 20 hours of work.
Report length	4 to 6 pages as PDF

Data we provide

You will receive a single zip containing:

- `market_states.parquet` — 60 trading days of NIFTY options data at 30-minute intervals (~900 rows), covering days 1–60. Days 1–30 are the training window. Days 31–60 are the evaluation window.

- `finetune_instructions.jsonl` — 300 instruction-format training samples drawn from the training window. Inspect before use.
- `rag_corpus.jsonl` — 100 historical market episode summaries for retrieval context.
- `retrieve.py` — The provided retrieval function. Do not modify it. Place it in the same directory as rag_corpus.jsonl.

A deliberate note on the instruction file: we have not documented exactly how this file was constructed. Treat it as you would any dataset arriving from a real third-party pipeline. Not all examples in the file are safe to use as provided. Part of your job is to figure out which examples you trust, what is wrong with those you do not, and what to do about it before training begins.

Data you may not use

To keep submissions comparable, do not add any data from outside the provided bundle: no broker APIs, no NSE bhavcopy, no news or sentiment, no alternative data sources. Do not modify the retrieve.py function or replace it with a different retrieval implementation. If you would want additional data or a different retrieval strategy in a real setting, mention it in your report with a short justification, but do not incorporate it into your submission.

Deliverables

Submit three things, in this order of importance:

- **Written report.** PDF, 4 to 6 pages, structured per the section prompts below. This is the most heavily weighted artefact.
- **Code repository.** Private GitHub repo containing your fine-tuning notebook, orchestrator implementation, MLflow artifacts, LoRA adapter weights or a download link, requirements.txt or pyproject.toml, and a README with setup plus a run command. Eval suite code must be committed before your first training run. Kaggle notebook URL in the README.
- **Live walkthrough.** A 20-minute session with us in the final round. We will ask you to navigate and modify your code in real time. Every candidate will be asked: ‘If this orchestrator were receiving signals from three pods simultaneously, what would you change in the design?’

The report: structure and prompts

The report carries more weight than the code. A working pod paired with a thoughtful report will outscore a technically impressive submission with a thin writeup. Address every section below.

1. Eval suite design (around 1 page)

Write this section before you begin training and commit it to your repository before your first Kaggle run. Define the complete set of metrics and conditions you will use to determine whether your pod is trustworthy enough to connect to the orchestrator. Your eval suite must include at minimum: walk-forward directional accuracy per 5-day window, output schema pass rate, conviction validity analysis (not just a reliability diagram — explain what makes conviction a meaningful field in your specific implementation), orchestrator suppression and downgrade rates by window, and performance broken down across high-VIX versus low-VIX regimes. State specific threshold values before you know your results: what numbers would constitute a trustworthy pod, and what numbers would constitute a failing one.

2. Data audit (around 1 page)

This is the section we care about most. Walk through what you found in the instruction-format training file — not the categories of check you ran, but what it actually contained, specifically. For each finding: what is it, where is it, what did you decide to do about it, and why that decision over the alternatives? If something looked anomalous and you investigated, describe what it took to figure out what you were looking at. If your inspection found nothing wrong with the file, explain in precise technical detail why you are confident it is clean. We will be sceptical of that answer.

3. Fine-tuning and RAG (around 2 pages)

Document your model choice and the reasoning behind it. Show your complete instruction template with one worked example: the full input market state, the constructed prompt, and the expected output JSON. Walk through your LoRA configuration and justify each parameter choice. Describe how you handled any issues found in the training data and how you verified the fix. Address the conviction field as a design problem: where does the value come from in your fine-tuned model, why softmax over the direction token is not sufficient, and what you did to make the value informative. Include a table of your MLflow runs showing what changed between experiments and why. Paste your Kaggle notebook URL.

For the RAG experiment: describe your prompt template design, report the two-condition ablation results across the full walk-forward evaluation set, and explain whether retrieval changed signal quality and in which direction. Examine specifically whether retrieved context changed conviction scores and whether those changes were directionally justified by the similarity of the retrieved episodes to the current market state.

4. Results (around 1 page)

Report your results against the eval suite you defined in Section 1. Walk-forward directional accuracy per 5-day window. Output schema pass rate. Conviction reliability across bins. Orchestrator suppression rate, low-conviction downgrade rate, and parse failure rate across the evaluation set — and separately for windows where the VIX spike occurred. Confidence intervals on every metric, not point estimates. Compare your results against the thresholds you committed to in Section 1 and explain any gaps.

5. How do I know this pod is safe to connect? (at least 1 page — critical)

Answer the following question with reference to your specific implementation:

> It is 09:30 on an expiry Thursday. India VIX has opened 3σ above its trailing 30-day mean and ADX is reading 14. Walk through what happens in your system from market state ingestion to final orchestrator output. At each stage — regime check, model inference, schema validation, conviction threshold — state exactly what your implementation does, what values it produces, and what gets logged. Then state what is wrong with your current implementation for this class of event and what you would add to handle it correctly.

Address, at minimum: specific ways your fine-tuning and eval suite could have missed something, what data condition would expose that gap, and whether your orchestrator suppression rates look right to you — which windows show elevated suppression and why. A candidate who writes 'the orchestrator would suppress the signal, therefore the pod is safe' will not advance. A candidate who writes 'the orchestrator suppresses correctly, but I am not yet confident at these specific boundaries, here is why, and here is what I would build next' is exactly who we want.

Evaluation rubric

We score on five dimensions. Weights are explicit and final.

Dimension	Weight	What we are assessing
Fine-tuning discipline	25%	Instruction data inspected and issues handled before training. LoRA configuration reasoned, not default. Conviction field explicitly designed and validated. MLflow tracking complete from run one.
Agentic orchestration	25%	All suppression rules implemented correctly. Reason codes logged on every decision. Wrapper output structure correct. Walk-forward set run through the full orchestrator with suppression and downgrade rates reported by window.
Eval suite design (pre-training)	20%	Specific thresholds committed before training. Conviction validity addressed as a design problem, not just a metric. Regime-sliced evaluation included.
RAG experiment	15%	Ablation run correctly across the full walk-forward set. Conviction change under retrieval examined and explained. Result reported honestly.

Analysis and writeup	15%	Depth of Section 5. Honest identification of what the current system gets wrong. Concrete proposals for what to build next.

Note on the weightings: fine-tuning discipline and agentic orchestration together carry 50% of the marks. We can teach RAG pipeline design. We cannot easily teach the instinct to inspect data you have been handed, or to design systems that fail safely. A submission that finds a real issue in the training data, reports orchestrator behaviour honestly, and produces a modest but credible signal will beat a polished submission with fabricated eval numbers, every time.

Rules and constraints

- Open-source libraries. You may use any open-source library. Document every dependency in your requirements file.
- Compute. All model training must be run on Kaggle free-tier GPU. We will verify Kaggle execution timestamps against your MLflow run timestamps.
- LLM assistance. You may use ChatGPT, Claude, Copilot, Cursor, or any other tool for coding help. We expect this. However, in the live walkthrough we will ask you to navigate and modify your own code — if you cannot, that is a strong negative signal regardless of how polished the submission looks.
- Walk-forward only. k-fold cross-validation on a time series is disqualifying.
- Eval suite first. Your eval suite code must be committed to your repository before your first training run begins. Section 1 must contain specific threshold values, not just metric names.
- Fixed schema and orchestrator logic. Do not modify the signal field names, permitted values, or the suppression rules defined in this brief.
- Independent work. You may discuss the problem conceptually with peers, but all code and writeup must be entirely your own.
- No external data. Do not use any data beyond what we provide. Do not modify or replace the retrieve.py function.

Submission

Email a single link to your private GitHub repository to the address provided in your invitation, with subject line **AI-SLM Screening — [Your Name]**. The repository must contain your code, MLflow artifacts, LoRA adapter weights or download link, README, and the PDF report. Deadline: as stated in your invitation email, in IST. Late submissions will not be reviewed.

A final note

The ambiguities and omissions in this brief are intentional. We are not trying to catch you out for sport. We are trying to see how you handle a realistic AI systems setup, which is never cleanly

specified. A careful submission that finds a real issue in the training data, describes a correctly-functioning orchestrator, and reports a modest but honest result will always outscore a submission that papers over problems and claims strong performance.

The AI pods in this system operate under one constraint above all others: AI earns its place only after the deterministic layer has proven edge. Pod outputs are aggregated, scrutinised, and held to high evidential standards before influencing any decision. We are looking for researchers who build evals before they claim results. Write accordingly.

Direct your questions to: surya@quantsingularity.in Good luck. We look forward to reading what you build.

Surya

Quant Singularity